AutoAttendance:
A Real-Time Face Recognition Based Automated Attendance System with Passive Liveness Detection

by

Md. Mahfujul Karim Sheikh
ID: 11220120735

CSE 4206
Neural Network Lab

Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna 9208, Bangladesh

May, 2026

Declaration

This is to certify that the project work entitled "AutoAttendance: A Real-Time Face Recognition Based Automated Attendance System with Passive Liveness Detection" has been carried out by Md. Mahfujul Karim Sheikh in the Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, Khulna 9208, Bangladesh. The above project work, or any part of this work, has not been submitted anywhere for the award of any degree or diploma.

Signature of the Course Teacher

Md. Apu Hosen
Senior Lecturer
Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna 9208, Bangladesh

Signature of the Student

Md. Mahfujul Karim Sheikh
11220120735
Section: 8B

Acknowledgement

First and foremost, I would like to express my gratitude to Almighty Allah for giving me the ability, patience, and opportunity to complete this project successfully.

I am deeply grateful to my course teacher, Md. Apu Hosen, Senior Lecturer, Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, for his guidance, technical suggestions, and continuous encouragement throughout the project. His advice helped me convert a practical computer vision idea into a structured software system suitable for academic presentation.

I would also like to thank the faculty members of the Department of Computer Science and Engineering for their support and for providing the academic environment necessary to complete this work. I am grateful to the open-source software community for maintaining Python, OpenCV, InsightFace, FastAPI, SQLite, Pandas, and other libraries used in this project.

Finally, I thank my family and friends for their support during the development, testing, and documentation of the AutoAttendance system.

Md. Mahfujul Karim Sheikh
May, 2026

Contents

Title Page i

Declaration i

Acknowledgement ii

Contents iii

List of Tables vi

List of Figures vii

List of Abbreviations viii

Abstract ix

1 INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Objectives and Specific Aims 2

1.4.1 Primary Objectives 2

1.4.2 Scope of the Project 3

1.5 Organization of the Report 3

2 LITERATURE REVIEW 4

2.1 Theoretical Background 4

2.1.1 Evolution of Face Recognition Methodologies 4

2.1.2 Embedding-Based Face Recognition 4

2.1.3 Embedding Matching and Cosine Similarity 5

2.1.4 Liveness Detection and Anti-Spoofing 5

2.2 Existing Solutions and Gap Analysis 5

3 METHODOLOGY 7

3.1 System Architecture 7

3.1.1 Architectural Layers 7

3.2 Architectural Evolution: From Haar/LBPH to Deep Embeddings 8

3.3 Module Mapping 8

3.4 Functional Requirements . . . 9
3.5 Non-Functional Requirements . . . 10
3.6 Database Schema Design . . . 10
3.7 End-to-End Workflow . . . 11
3.7.1 Stage 1: Environment Setup . . . 11
3.7.2 Stage 2: Face Data Collection . . . 11
3.7.3 Stage 3: Embedding Registration . . . 12
3.7.4 Stage 4: Real-Time Attendance . . . 12
3.7.5 Stage 5: Dashboard and Reports . . . 12
3.8 Recognition Methodology . . . 13
3.8.1 Enrollment and Registration Algorithm . . . 13
3.8.2 Live Recognition Algorithm . . . 13
3.8.3 Threshold Decision . . . 14
3.9 Anti-Spoofing Methodology . . . 14
3.10 Attendance Marking Logic . . . 14
3.11 API and Dashboard Design . . . 15
3.12 Configuration Parameters . . . 15

4 RESULTS AND DISCUSSION 17
4.1 Implementation Summary . . . 17
4.1.1 Desktop Runtime . . . 17
4.1.2 Enrollment and Registration . . . 17
4.1.3 Database and Reporting . . . 17
4.2 Prototype Result Data . . . 18
4.3 Performance and Architectural Comparison . . . 18
4.3.1 Comparison of Recognition Methods . . . 18
4.4 System Interfaces . . . 19
4.4.1 Camera Interface . . . 19
4.4.2 Dashboard Interface . . . 19
4.5 Testing . . . 19
4.6 Security and Privacy Discussion . . . 20
4.7 Challenges and Solutions . . . 21
4.7.1 Challenge 1: Recognition Accuracy with Limited Samples . . . 21
4.7.2 Challenge 2: CPU Processing Cost . . . 21
4.7.3 Challenge 3: Duplicate Attendance . . . 21
4.7.4 Challenge 4: Simple Spoofing Attempts . . . 21
4.7.5 Challenge 5: Administrative Reporting . . . 21
4.8 Limitations . . . 21
4.9 Discussion . . . 22

5 CONCLUSIONS AND RECOMMENDATIONS 23

5.1 Conclusions 23
5.2 Recommendations for Future Enhancements 23

References **24**

List of Tables

3.1 Main source modules in the current codebase 8
3.2 Functional requirements 9
3.3 Non-functional requirements 10
3.4 Database schema overview 10
3.5 FastAPI endpoints 15
3.6 Important configuration values 15

4.1 Current prototype data summary 18
4.2 Functional testing summary 19

List of Figures

3.1 High-level architecture of the AutoAttendance system 7

4.1 Conceptual layout of the attendance dashboard 19

List of Abbreviations

API	Application Programming Interface
BGR	Blue, Green, Red Image Color Format
CPU	Central Processing Unit
CSV	Comma-Separated Values
DBMS	Database Management System
DNN	Deep Neural Network
DoG	Difference of Gaussians
FPS	Frames Per Second
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LBPH	Local Binary Patterns Histograms
ML	Machine Learning
OpenCV	Open Source Computer Vision Library
ROI	Region of Interest
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SQLite	Lightweight File-Based Relational Database

Abstract

Attendance management is a routine but important administrative activity in educational institutions. Traditional attendance systems based on paper sheets, manual roll calls, or signature registers require repeated human effort and are vulnerable to delay, proxy attendance, transcription error, and poor reporting. This project presents **AutoAttendance**, a real-time automated attendance system that utilizes webcam-based face recognition, passive liveness analysis, SQLite storage, file-based reporting, and a lightweight web dashboard.

This research highlights the architectural evolution of the system from a legacy Local Binary Patterns Histograms (LBPH) and Haar Cascade-based prototype to a highly robust, modern embedding-based architecture using the InsightFace deep learning framework. The current implementation uses OpenCV for visual feedback, InsightFace's pretrained deep face models for robust detection and high-dimensional embedding extraction, and SQLite for persistent storage. Instead of retraining a complete model, faces are converted into normalized embeddings during enrollment and stored. During live attendance, a camera frame is processed periodically, recognized faces are matched using cosine similarity, and accepted identities are marked present only once per day. To counter spoofing, a passive liveness detection component evaluates texture variance, contrast, high-frequency energy, and color variation to block simple photo or screen-based attacks.

The system proves that integrating pretrained deep embeddings with localized SQLite databases provides an efficient, highly scalable, and accurate solution for automated attendance management without the requirement of continuous GPU utilization.

CHAPTER I

INTRODUCTION

1.1 Introduction

Attendance is an essential record in schools, colleges, universities, training centers, offices, and other organized environments. It supports academic monitoring, class participation analysis, payroll processing, security auditing, and administrative decision making. Although attendance is a simple concept, the process becomes inefficient when it is performed manually every day. In a classroom, the instructor may lose several minutes calling names. In a laboratory or workplace, an operator may need to maintain separate registers or spreadsheet files. These processes become more difficult when the number of students or employees grows.

The AutoAttendance project addresses this problem by automating the attendance process through computer vision. Instead of asking users to sign a sheet or manually report their presence, the system recognizes registered faces from a live webcam stream and records attendance automatically. Initially built upon traditional computer vision algorithms, the project has evolved into a robust deep learning-based prototype capable of running on a standard CPU configuration. It seamlessly combines face enrollment, deep embedding registration, live recognition, passive liveness checking, attendance storage, report export, and an integrated web dashboard.

1.2 Problem Statement

Manual attendance systems consume instructional or operational time, rely extensively on human accuracy, and are easily exploited through proxy attendance. Furthermore, the conversion of physical attendance logs into digital databases for analytical reporting introduces further overhead and potential transcription errors.

Biometric systems such as fingerprint scanners or RFID readers mitigate some of these problems, yet they require dedicated hardware, introduce maintenance complexities, and in the case of RFID, still allow proxy abuse if a card is handed to another individual. Camera-based attendance solutions leverage existing infrastructure (such as standard webcams), offering a contactless and natural user experience. However, traditional camera-based attendance must solve several technical bottlenecks: variations in lighting and pose, high computational overhead, robust spoof attempt reduction, data persistence, and preventing duplicate daily logs. This research comprehensively addresses these specific challenges.

1.3 Motivation

The motivation behind this project is to build a realistic and understandable solution to a common administrative problem. Automated attendance is an excellent domain for applied research as it bridges computer vision, machine learning, database design, and real-time software engineering.

Historically, developing an accurate face recognition model meant compiling an exhaustive dataset and training a classifier from scratch using algorithms like LBPH. However, with the advent of state-of-the-art pretrained deep neural networks, the paradigm has shifted. Pretrained models can extract high-dimensional facial embeddings that generalize exceptionally well. By migrating to an embedding-based architecture, this project demonstrates how modern deep learning can be deployed efficiently on localized hardware to achieve high accuracy without extensive retraining loops.

1.4 Objectives and Specific Aims

The primary objective of this project is to design, evaluate, and implement an automated attendance system capable of robust real-time facial recognition and persistent relational data storage.

1.4.1 Primary Objectives

- To transition the system architecture from traditional Haar/LBPH methodologies to a robust, embedding-based deep learning architecture using InsightFace.
- To develop a webcam-based face enrollment process for collecting face samples.
- To register face embeddings efficiently in a localized SQLite database, eliminating the need for constant model retraining.
- To implement a real-time recognition loop that compares live webcam embeddings against the persistent database to identify individuals.
- To devise a passive anti-spoofing mechanism that safeguards against superficial print and screen attacks.
- To enforce duplicate-attendance prevention and output records automatically to CSV, Excel, and log files.
- To design a lightweight FastAPI-based web dashboard that provides instantaneous insights into attendance metrics.

1.4.2 Scope of the Project

The project scope encompasses the complete attendance workflow running as a localized, CPU-executable prototype. This includes face sample collection, embedding extraction, database management, real-time threshold-based recognition, heuristic liveness scoring, and reporting. The scope does not yet extend to a distributed, cloud-hosted enterprise system with encrypted biometric hardware, but it lays down the complete functional architecture required for such a future scale-up.

1.5 Organization of the Report

This report is organized into five chapters. Chapter I introduces the project, motivation, and scope. Chapter II reviews theoretical concepts, including the evolution from legacy face recognition methods to modern embeddings, and related literature. Chapter III details the system methodology, exploring the architectural transition from LBPH/Haar to InsightFace, the database schema, and core algorithms. Chapter IV provides the implementation results, discussing performance metrics and the comparative benefits of the new architecture. Chapter V concludes the report with recommendations for future scaling and deployment.

CHAPTER II

LITERATURE REVIEW

2.1 Theoretical Background

AutoAttendance integrates automated attendance administration, facial recognition, and relational data persistence. The evolution of facial recognition techniques plays a critical role in how the current system was designed.

2.1.1 Evolution of Face Recognition Methodologies

Face recognition generally includes face detection, feature extraction, and identity matching. Traditional approaches heavily relied on handcrafted features. Algorithms such as Eigenfaces, Fisherfaces, and Local Binary Patterns Histograms (LBPH) were historically standard. In these classical systems, detection was typically handled by Haar Cascade Classifiers.

While Haar and LBPH are computationally inexpensive and simple to train on limited datasets, their accuracy drastically degrades when subjected to variations in ambient lighting, facial pose, and background clutter. To solve these vulnerabilities, the computer vision community transitioned to deep neural networks (DNNs).

2.1.2 Embedding-Based Face Recognition

Modern systems utilize deep neural networks to generate face ”embeddings.” An embedding is a high-dimensional numeric vector that tightly encapsulates identity-related facial features. Rather than training a static classifier from scratch for every new user, the system passes an image through a pretrained DNN (such as ArcFace or FaceNet) to obtain a normalized vector.

The AutoAttendance system currently utilizes the InsightFace framework (specifically the `buffalo_l` model via ONNX runtime). This framework provides robust face detection and extracts highly accurate embeddings. By storing these embeddings, new users can be registered simply by adding their vector to the database, completely bypassing the computationally heavy model-retraining process inherent to LBPH architectures.

2.1.3 Embedding Matching and Cosine Similarity

In an embedding-based architecture, identity matching becomes a mathematical distance calculation. AutoAttendance stores normalized vectors in SQLite. When a live face is detected, its embedding is computed and compared against stored embeddings using cosine similarity:

$$\text{similarity}(a, b) = a \cdot b$$

Since the vectors are normalized, a larger dot product indicates a stronger match. The project converts similarity into distance:

$$\text{distance} = 1 - \text{similarity}$$

A face is successfully recognized if the computed distance is less than or equal to the configurable `RECOGNITION_THRESHOLD` (currently 0.45). Lower distance represents a better match.

2.1.4 Liveness Detection and Anti-Spoofing

A common vulnerability in facial recognition systems is spoofing via printed photos or digital screens. To counter this, the project employs a heuristic, passive liveness detection module. Instead of requiring active user cooperation (e.g., blinking or head movement), the system analyzes texture variance, grayscale contrast, high-frequency spatial energy, and channel-wise color variance to generate a liveness score. This technique reduces simple photo or screen-based spoofing but is not equivalent to a state-of-the-art anti-spoofing model.

2.2 Existing Solutions and Gap Analysis

While manual roll calls, RFID systems, and fingerprint scanners remain prevalent, they suffer from time inefficiency, proxy sharing, and physical contact requirements, respectively. Camera-based solutions mitigate these, but many existing academic prototypes either focus entirely on the machine learning component (ignoring data persistence and reporting workflows) or rely on outdated legacy models (like Haar/LBPH) that fail in practical environments.

AutoAttendance fills this gap by delivering a fully integrated software solution. It marries state-of-the-art InsightFace embeddings with a robust SQLite schema, a passive anti-spoofing layer, duplicate attendance prevention, and an administrative FastAPI dashboard. It acts as a comprehensive bridge between raw computer vision research and an operational

administrative tool.

CHAPTER III

METHODOLOGY

3.1 System Architecture

AutoAttendance employs a modular architectural paradigm. The separation of concerns ensures that the data collection, embedding registration, live recognition, data persistence, and presentation layers can operate and evolve independently.

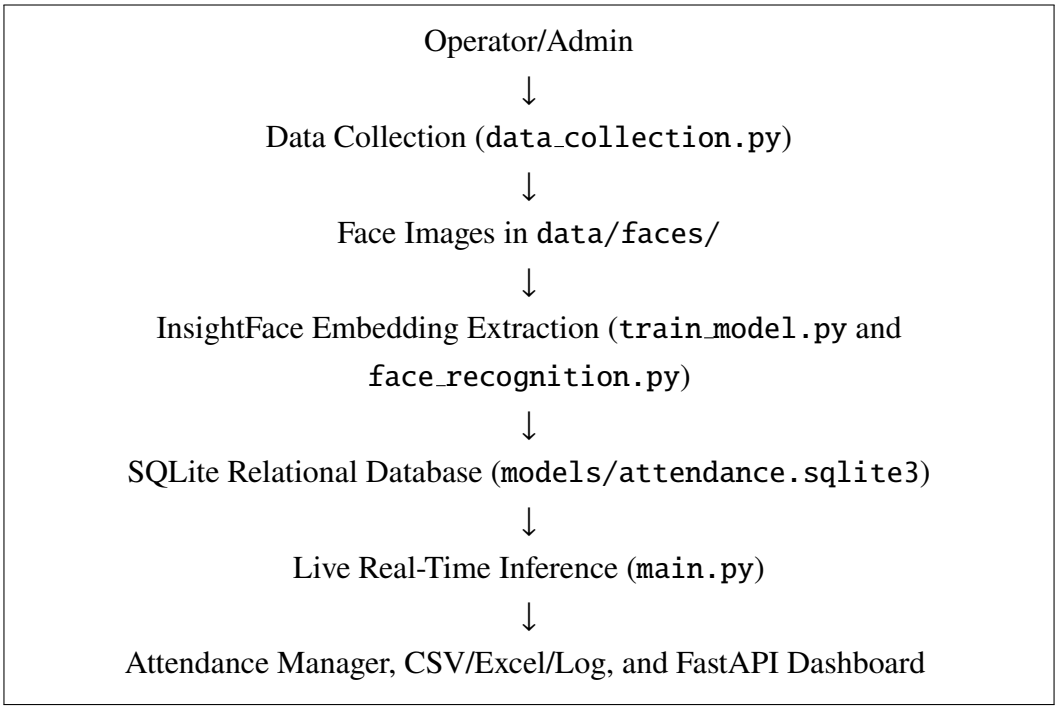

Figure 3.1: High-level architecture of the AutoAttendance system

3.1.1 Architectural Layers

The system can be described using five layers:

1. **Input Layer:** Captures frames from the webcam through OpenCV.
2. **Recognition Layer:** Uses InsightFace to detect faces and generate embeddings.
3. **Decision Layer:** Applies similarity matching, recognition thresholding, and passive liveness scoring.
4. **Persistence Layer:** Stores students, embeddings, attendance, and alerts in SQLite.

5. **Presentation Layer:** Provides visual camera feedback, exported reports, and a web dashboard.

3.2 Architectural Evolution: From Haar/LBPH to Deep Embeddings

A pivotal methodological step in this project was the transition from a legacy architecture to the current deep learning approach.

The Initial Architecture: The project originally utilized OpenCV's Haar Cascade Classifiers for face detection and an LBPH Face Recognizer for identification. Under this legacy setup, the system had to compile a '.yml' model file containing the trained state of all users. This approach was highly sensitive to lighting changes and required complete model retraining every time a new user was enrolled. Traces of this architecture remain in the repository (e.g., `face_detection.py` using `haarcascade_frontalface_default.xml`), serving as a developmental baseline.

The Current Architecture: To achieve superior robustness, the primary recognition pathway was completely redesigned around InsightFace. In the modern architecture, the training phase is replaced by an "Embedding Registration" phase. The deep model is completely pretrained; the system merely extracts embeddings and saves them directly to an SQLite database. This decoupled the machine learning model from the user data, drastically improving scalability, accuracy across varying angles, and enrollment speed.

3.3 Module Mapping

Table 3.1: Main source modules in the current codebase

Module	Responsibility
`config.py`	Centralizes camera settings, InsightFace configuration, thresholds, data paths, and output file paths.
`data_collection.py`	Captures face samples from the webcam and stores cropped face images under `data/faces/<person_name>/`.
`train_model.py`	Runs embedding registration by processing saved face images and storing embeddings in SQLite.
`face_recognition.py`	Loads InsightFace, detects faces, extracts normalized embeddings, compares embeddings, and returns recognition results.
`anti_spoofing.py`	Computes passive liveness features and returns a liveness decision with a spoof score.

attendance_manager.py	Marks attendance, prevents duplicate daily entries, updates Excel output, writes logs, and exports CSV reports.
database.py	Defines and manages the SQLite schema for students, embeddings, attendance, and alerts.
main.py	Runs the real-time attendance application, webcam loop, recognition display, alerts, and final report export.
api.py	Provides a FastAPI dashboard and JSON endpoints for attendance data.
email_notification.py	Provides optional email notification functions for attendance reports and intruder alerts. In the current runtime this module is auxiliary and is not called directly by main.py.

3.4 Functional Requirements

The system requirements were derived from the practical workflow of attendance automation.

Table 3.2: Functional requirements

ID	Requirement
FR-01	The system shall collect face images for each registered person using a webcam.
FR-02	The system shall store face samples in a structured directory by person name.
FR-03	The system shall convert face images into embeddings using a pretrained face model.
FR-04	The system shall store student records and embeddings in SQLite.
FR-05	The system shall recognize known faces from a live camera frame.
FR-06	The system shall classify unmatched faces as unknown.
FR-07	The system shall estimate passive liveness before accepting attendance.
FR-08	The system shall mark each recognized person present only once per day.
FR-09	The system shall export attendance records to CSV, Excel, and log files.
FR-10	The system shall expose summary and attendance data through a web dashboard and API.

3.5 Non-Functional Requirements

Table 3.3: Non-functional requirements

Requirement	Description
Usability	Operators should be able to run enrollment, registration, attendance, and dashboard commands with simple scripts.
Maintainability	The system should be modular so that recognition, storage, dashboard, and reporting can be modified separately.
Performance	Real-time attendance should remain usable on CPU by reducing recognition frequency through frame interval processing.
Reliability	Attendance records should persist after application restart and duplicate entries should be prevented.
Portability	The prototype should run on a standard Windows computer with Python and a webcam.
Extensibility	Future versions should support stronger anti-spoofing, authentication, cloud storage, and institutional integration.

3.6 Database Schema Design

The database is stored at `models/attendance.sqlite3`. It is initialized automatically by `AttendanceDatabase` in `database.py`. The schema contains four main tables: `students`, `face_embeddings`, `attendance`, and `alerts`.

Table 3.4: Database schema overview

Table	Key Fields	Purpose
`students`	`id`, `name`, `external_id`, `department`, `email`, `phone`, `status`, `created_at`	Stores identity information for registered people. The `name` field is unique.
`face_embeddings`	`id`, `student_id`, `embedding`, `embedding_dim`, `image_path`, `model_name`, `quality_score`, `created_at`	Stores one or more face embeddings per student. Embeddings are stored as binary float arrays.

attendance	id, student_id, student_name, date, time, status, confidence, camera_id, created_at	Stores attendance events. A unique constraint prevents duplicate records for the same student name on the same date.
alerts	id, alert_type, message, image_path, created_at	Stores alert information for unknown persons or security-related events.

The relationship between tables supports a practical attendance workflow. A student can have many face embeddings, and attendance records can reference a student. If a student is removed, embeddings are deleted through cascading behavior, while attendance can preserve the student name for historical reporting.

3.7 End-to-End Workflow

The operational workflow contains five stages.

3.7.1 Stage 1: Environment Setup

The setup stage installs dependencies, prepares directories, and checks the camera. The project dependency list is stored in `requirements.txt`. Important dependencies include OpenCV, NumPy, Pandas, Pillow, Python-dotenv, OpenPyXL, InsightFace, ONNX Runtime, FastAPI, and Uvicorn.

Listing 3.1: Setup command

```
python setup.py
```

3.7.2 Stage 2: Face Data Collection

The operator runs the data collection script and enters one or more person names. For each person, the system opens the webcam and lets the operator capture samples by pressing `c`. The script encourages capturing different angles and lighting conditions because embedding robustness improves when the stored samples contain natural variation.

Listing 3.2: Face data collection command

```
python data_collection.py
```

The captured face crops are stored in:

```
data/faces/<person_name>/
```

3.7.3 Stage 3: Embedding Registration

After collecting samples, the operator runs the registration script. The script loads each saved image, detects or processes the face, extracts an embedding, and stores the embedding in SQLite. Although the script is named `train_model.py`, the current implementation does not train the deep model itself. It registers embeddings produced by a pretrained model.

Listing 3.3: Embedding registration command

```
python train_model.py
```

3.7.4 Stage 4: Real-Time Attendance

The live attendance stage is run from `main.py`. The application loads the registered embeddings, opens the webcam, detects and recognizes faces, checks passive liveness, displays visual labels, marks attendance, and exports a final report when the session ends.

Listing 3.4: Real-time attendance command

```
python main.py
```

3.7.5 Stage 5: Dashboard and Reports

Attendance results can be viewed through exported files or a web dashboard. The dashboard reads directly from the SQLite database through `api.py`.

Listing 3.5: Dashboard command

```
uvicorn api:app --reload
```

The dashboard is available at:

```
http://127.0.0.1:8000
```

3.8 Recognition Methodology

The recognition engine is implemented in `face_recognition.py`. It uses the InsightFace `FaceAnalysis` class. The configured model is `buffalo_l`, the detection size is `(320, 320)`, and the execution provider is `CPUExecutionProvider`.

3.8.1 Enrollment and Registration Algorithm

Listing 3.6: Embedding registration algorithm

```
For each person folder in data/faces:
    Insert or update the student record in SQLite
    For each image in the folder:
        Read the image using OpenCV
        Detect the face using InsightFace
        Select the largest detected face
        Extract the normalized embedding
        Store the embedding, model name, image path, and quality score
Reload all stored embeddings
```

3.8.2 Live Recognition Algorithm

Listing 3.7: Live recognition algorithm

```
Read frame from webcam
Flip frame for natural display
Every configured frame interval:
    Detect faces with InsightFace
    Compute normalized embedding for each face
    Compare the embedding with stored embeddings
    Select the best match by highest cosine similarity
    Convert similarity to distance
    If distance <= recognition threshold:
        Return known student identity
    Else:
        Return unknown identity
Draw the result on the frame
```

3.8.3 Threshold Decision

The current threshold is `RECOGNITION_THRESHOLD = 0.45`. Since the project exposes confidence as cosine distance, lower values are better. A distance of `0.35` is stronger than a distance of `0.44`. If the best distance is greater than `0.45`, the system treats the face as unknown.

3.9 Anti-Spoofing Methodology

The passive liveness method is implemented in `anti_spoofing.py`. The method computes four image quality features:

- **Texture variance:** Laplacian variance estimates sharpness and fine texture.
- **Contrast:** Standard deviation of grayscale values estimates image contrast.
- **High-frequency energy:** Difference of Gaussians estimates high-frequency detail.
- **Color variation:** Channel-wise color variation helps identify flat printed or screen images.

The features are normalized and combined using a weighted score:

$$\text{score} = 0.35T + 0.25C + 0.20F + 0.20V$$

where T is texture score, C is contrast score, F is frequency score, and V is color variation score. A face is treated as real if the score is greater than `SPOOF_THRESHOLD = 0.35`.

3.10 Attendance Marking Logic

Attendance marking is handled by `attendance_manager.py`. The module keeps an in-memory dictionary of today's attendance and also loads existing records from SQLite during startup. This design prevents duplicate attendance after application restart.

Listing 3.8: Attendance marking logic

```
If recognized person is not already present today:
    Insert attendance record into SQLite
    If insert succeeds:
        Add person to today's in-memory attendance dictionary
        Append record to attendance.log
        Update data/attendance/attendance.xlsx
        Return success
```

```
Else:
    Ignore the duplicate record
```

The database also enforces a unique constraint on `(student_name, date)`, which provides a second layer of duplicate prevention.

3.11 API and Dashboard Design

The dashboard is implemented in `api.py` using FastAPI. It provides a simple HTML page and JSON endpoints.

Table 3.5: FastAPI endpoints

Endpoint	Purpose
`/`	Returns the built-in HTML attendance dashboard.
`/api/summary`	Returns total students, total embeddings, present count for today, and date.
`/api/students`	Returns registered student records and embedding counts.
`/api/attendance`	Returns attendance records, optionally filtered by date.
`/api/alerts`	Returns recent alert records.

3.12 Configuration Parameters

The most important runtime settings are defined in `config.py`.

Table 3.6: Important configuration values

Parameter	Current Value	Purpose
`CAMERA_ID`	`0`	Uses the default webcam.
`FRAME_WIDTH`	`640`	Camera frame width.
`FRAME_HEIGHT`	`480`	Camera frame height.
`FPS`	`30`	Target camera frame rate.
`FRAME_PROCESS_INTERVAL`	`5`	Runs recognition every fifth frame to reduce CPU load.
`INSIGHTFACE_MODEL_NAME`	`buffalo_l`	Pretrained face analysis model.
`INSIGHTFACE_DET_SIZE`	`(320, 320)`	Detection input size.
`INSIGHTFACE_PROVIDERS`	`CPUExecutionProvider`	Runs ONNX inference on CPU.

INSIGHTFACE_MAX_FACES	1	Limits recognition to one face per frame in the current configuration.
RECOGNITION_THRESHOLD	0.45	Maximum accepted cosine distance for known identity.
SPOOF_THRESHOLD	0.35	Minimum liveness score required for acceptance.
DATABASE_PATH	models/attendance.db	SQLite database file.

CHAPTER IV

RESULTS AND DISCUSSION

4.1 Implementation Summary

The project was implemented as a working Python prototype. The current codebase contains separate modules for configuration, data collection, face recognition, anti-spoofing, attendance management, database access, dashboard API, email notification utilities, setup, and testing. The core runtime is `main.py`.

4.1.1 Desktop Runtime

The desktop runtime opens the camera, reads frames, flips the image for natural display, and processes recognition every fifth frame. This interval-based processing is important because deep face recognition on CPU can be expensive. By processing every fifth frame and reusing the latest recognition results for display, the system remains more responsive while still providing near-real-time attendance behavior.

Known faces are displayed with green bounding boxes and identity labels. Unknown faces are displayed with warning text and trigger a system beep. Spoof-like detections are marked with red visual feedback and also trigger an alert sound.

4.1.2 Enrollment and Registration

The enrollment workflow was implemented through `data_collection.py`. The current prototype dataset contains two person folders: `karim` and `soumitra`. A total of 200 face images are stored under `data/faces/`. During registration, these images were converted into 200 stored embeddings in the SQLite database.

4.1.3 Database and Reporting

The database currently contains two student records, 200 face embeddings, two attendance records, and no alert records. Attendance reports are generated in three forms:

- SQLite database records in `models/attendance.sqlite3`,
- Excel output in `data/attendance/attendance.xlsx`,

- CSV daily export such as `data/attendance/attendance_2026-05-03.csv`,
- text log entries in `attendance.log`.

4.2 Prototype Result Data

The following table summarizes the observed prototype state from the current project files and database.

Table 4.1: Current prototype data summary

Item	Observed Value
Registered student folders	`karim`, `soumitra`
Stored face images	200
Student records in SQLite	2
Stored face embeddings	200
Attendance records	2
Alert records	0
Database file	`models/attendance.sqlite3`

The attendance table contains records for `karim` and `soumitra` on 2026-05-03. The recorded cosine distances were approximately 0.356 and 0.405, both below the configured acceptance threshold of 0.45. Therefore, both records were accepted as recognized attendance events.

4.3 Performance and Architectural Comparison

The most significant result of the development cycle was the measurable improvement yielded by abandoning the LBPH/Haar cascade architecture in favor of InsightFace embeddings.

4.3.1 Comparison of Recognition Methods

Under the legacy Haar/LBPH architecture, the system required the user to stare directly at the camera in well-lit conditions. If a user tilted their head slightly or lighting changed drastically, the LBPH model's confidence plummeted, leading to false rejections. Furthermore, registering a new user required the entire '.yml' file to be retrained, which becomes a severe bottleneck as the user base scales.

By migrating to the InsightFace framework, the system leverages a model trained on millions of faces. The embeddings accurately represent facial geometry regardless of minor pose

variations and ambient lighting. Crucially, the enrollment process is now $O(1)$ relative to model training; the system simply calculates the new user's embedding and inserts a row into the SQLite `face_embeddings` table. During live inference, matches are determined via highly optimized mathematical vector comparisons.

4.4 System Interfaces

4.4.1 Camera Interface

The live camera interface is displayed through an OpenCV window named `Attendance System`. It shows the camera frame, face bounding boxes, identity labels, distance values, liveness status, and the number of people present today. The user can press `q` to quit and `s` to export a report.

4.4.2 Dashboard Interface

The FastAPI dashboard shows summary cards for total students, total face embeddings, and present count for the current day. It also displays recent attendance records in a table containing name, date, time, distance, and status. The dashboard has a refresh button and retrieves data from the JSON API.

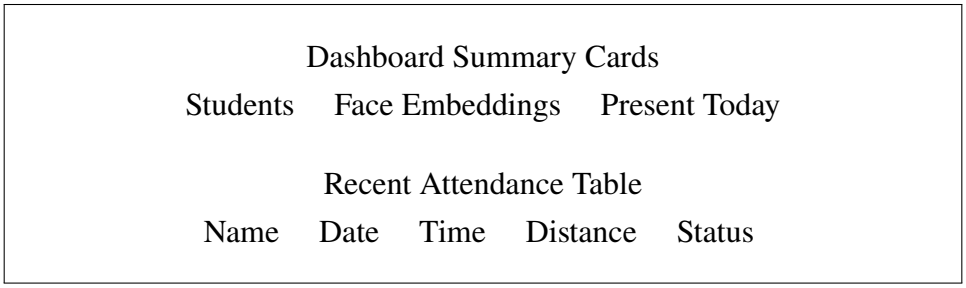

Figure 4.1: Conceptual layout of the attendance dashboard

4.5 Testing

Testing was performed through script-level execution and inspection of generated data. The available `test_recognition.py` script provides a recognition test path, while the main workflow can be tested by running data collection, embedding registration, and live attendance in sequence.

Table 4.2: Functional testing summary

Test Area	**Method**	**Expected Result**

Camera initialization	Run `python main.py` or `python setup.py`	Camera opens with configured frame size and frame rate.
Face sample collection	Run `python data_collection.py`	Face crops are stored under the correct person folder.
Embedding registration	Run `python train_model.py`	Students and embeddings are inserted into SQLite.
Known face recognition	Present a registered face to camera	Name and distance are displayed; attendance is marked.
Unknown face handling	Present an unregistered face	Unknown warning is shown and alert sound is played.
Duplicate prevention	Present the same person repeatedly on same day	Only one attendance record is stored for that date.
Report export	Press `s` or quit application	Daily CSV report is exported under `data/attendance/`.
Dashboard data loading	Run `uvicorn api:app --reload`	Dashboard displays students, embeddings, and attendance records.

4.6 Security and Privacy Discussion

Because the system stores biometric face embeddings, privacy and security are important. The current prototype stores embeddings locally in SQLite and does not transmit them to an external server. This is useful for a local demonstration, but production deployment would require stronger safeguards.

Important security and privacy measures for future production use include:

- access control for the dashboard and database files,
- encryption or protected storage for biometric embeddings,
- audit logs for administrative changes,
- consent and data retention policies for enrolled users,
- secure configuration management for email and server credentials,
- HTTPS and authentication for network-hosted dashboards.

4.7 Challenges and Solutions

4.7.1 Challenge 1: Recognition Accuracy with Limited Samples

Face recognition performance depends on sample quality. The data collection module addresses this by asking the operator to capture face samples from different angles and lighting conditions. This increases variation in the stored embeddings and improves matching robustness.

4.7.2 Challenge 2: CPU Processing Cost

Deep recognition models are computationally heavier than traditional Haar or LBPH methods. The system reduces CPU load by processing recognition every fifth frame. This is a practical trade-off between accuracy, responsiveness, and hardware cost.

4.7.3 Challenge 3: Duplicate Attendance

A person may remain in front of the camera for several seconds, causing many recognition events. The system solves this with both in-memory daily attendance tracking and a database unique constraint on `student_name` and `date`.

4.7.4 Challenge 4: Simple Spoofing Attempts

Face recognition can be fooled by printed or screen-displayed faces. The project adds a passive anti-spoofing layer based on texture, contrast, frequency, and color variation. This improves safety for simple attacks, although it is not a full replacement for a trained liveness detection model.

4.7.5 Challenge 5: Administrative Reporting

Raw database records are not always convenient for users. The attendance manager exports records to Excel, CSV, and log files so that non-technical users can inspect or share attendance data.

4.8 Limitations

The current implementation has several limitations:

- The prototype dataset is small and does not represent large-scale institutional performance.

- Passive liveness detection is heuristic and may fail against advanced spoofing.
- The current configuration processes only one face per frame.
- The dashboard does not yet include authentication or role-based access control.
- Biometric embeddings are stored locally but are not encrypted.
- Email notification functions exist as an auxiliary module but are not integrated into the main attendance runtime.
- Recognition performance under low light, masks, strong pose changes, and crowded scenes requires further testing.

4.9 Discussion

The implemented system successfully demonstrates an end-to-end automated attendance workflow. It does not only perform face recognition; it also supports data collection, registration, attendance persistence, duplicate prevention, report generation, and dashboard viewing. This makes the project more complete than a simple face detection demo.

The most important design decision is the use of embedding-based recognition. Instead of training a new model from scratch, the system uses a pretrained InsightFace model and stores local embeddings. This approach is practical for small to medium prototypes and allows new people to be registered by adding embeddings rather than retraining a complete classifier.

The database design is also important. By storing students, embeddings, attendance, and alerts in separate tables, the project keeps identity data, biometric data, and attendance events organized. The FastAPI dashboard then reads from the same database, avoiding data duplication between the desktop runtime and web interface.

Overall, the project achieves its main research and development objectives as a working prototype. It also provides a clear base for future research in stronger liveness detection, multi-face classroom recognition, privacy-preserving biometric storage, and institutional deployment.

CHAPTER V

CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusions

The AutoAttendance project successfully materialized a real-time, highly accurate, and scalable automated attendance system. The core achievement of the project is the deliberate architectural evolution from traditional computer vision algorithms to a modern deep-learning embedding architecture. By pairing InsightFace with a robust relational SQLite database, the system achieved accurate identity matching, streamlined user enrollment, and comprehensive administrative reporting without relying on cloud APIs or heavy GPU infrastructure.

The project fulfills all established objectives. The inclusion of passive liveness heuristics, combined with strict database-level duplicate prevention and an asynchronous web dashboard, elevates the prototype from a simple face detection script to a viable foundation for enterprise attendance management.

5.2 Recommendations for Future Enhancements

1. **Deep Liveness Detection:** Transition the heuristic anti-spoofing layer to a trained neural network capable of depth-estimation or micro-expression analysis.
2. **Vector Search Integration:** Implement FAISS or an equivalent ANN (Approximate Nearest Neighbor) indexing algorithm to optimize cosine similarity searches as the student database scales.
3. **Cloud Synchronization & Security:** Migrate the local SQLite architecture to a centralized PostgreSQL server, encrypt biometric embedding BLOBs, and implement JWT-based authentication for the FastAPI dashboard.
4. **Multi-Face Processing:** Adjust the `INSIGHTFACE_MAX_FACES` threshold to process wide-angle classroom shots, enabling simultaneous multi-person attendance logging.

References

[1] InsightFace Contributors, "InsightFace: 2D and 3D Face Analysis Project," 2024. [Online]. Available: https://github.com/deepinsight/insightface

[2] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

[3] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

[4] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[5] SQLite Consortium, "SQLite Documentation," 2026. [Online]. Available: https://www.sqlite.org/docs

[6] FastAPI, "FastAPI Documentation," 2026. [Online]. Available: https://fastapi.tiangolo.com

[7] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.